

HomeWork 07: Random Numbers, Monte Carlo Methods, and Empirical Distributions

Student Solution

May 11, 2026

1 Coin Tosses and the Law of Large Numbers

Method

Generate $N = 100000$ independent fair coin tosses. I encode heads as 1 and tails as 0. The running fraction of heads after n tosses is

$$\hat{p}_n = \frac{1}{n} \sum_{i=1}^n X_i, \quad (1)$$

where $X_i = 1$ for heads and $X_i = 0$ for tails. Since the coin is fair, $\mathbb{E}[X_i] = 0.5$. By the Law of Large Numbers,

$$\hat{p}_n \longrightarrow 0.5 \quad \text{as } n \rightarrow \infty. \quad (2)$$

Numerical output

Table 1: Running fraction of heads for selected values of n .

n	Running fraction of heads	Deviation from 0.5
10	0.700000	+0.200000
100	0.520000	+0.020000
1,000	0.508000	+0.008000
10,000	0.503500	+0.003500
100,000	0.498740	-0.001260

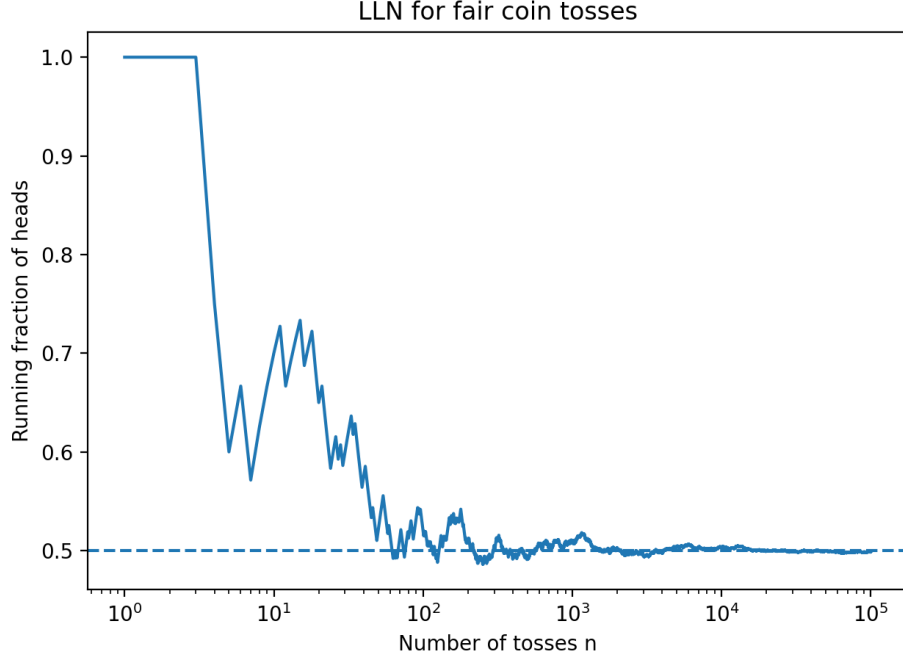


Figure 1: Running fraction of heads. The dashed line is the theoretical probability 0.5.

For small n , the running fraction fluctuates strongly. As n increases, the fraction becomes increasingly stable and approaches 0.5. This is exactly the behavior predicted by the Law of Large Numbers.

2 Monte Carlo Estimate of π

Method

To estimate π , generate independent random points (X, Y) uniformly in the unit square $[0, 1]^2$. The probability that a point falls inside the quarter unit circle is

$$\mathbb{P}(X^2 + Y^2 \leq 1) = \frac{\pi}{4}. \quad (3)$$

Therefore, if N_{in} is the number of points inside the quarter circle, the Monte Carlo estimator is

$$\hat{\pi}_N = 4 \frac{N_{\text{in}}}{N}. \quad (4)$$

Two independent random number generator streams are used: one for the x -coordinates and one for the y -coordinates.

Numerical output

Table 2: Monte Carlo estimates of π for several sample sizes.

N	N_{in}	$\hat{\pi}_N$	$ \hat{\pi}_N - \pi $
100	81	3.24000000	0.09840735
1,000	814	3.25600000	0.11440735
10,000	7,802	3.12080000	0.02079265
100,000	78,703	3.14812000	0.00652735
1,000,000	785,752	3.14300800	0.00141535

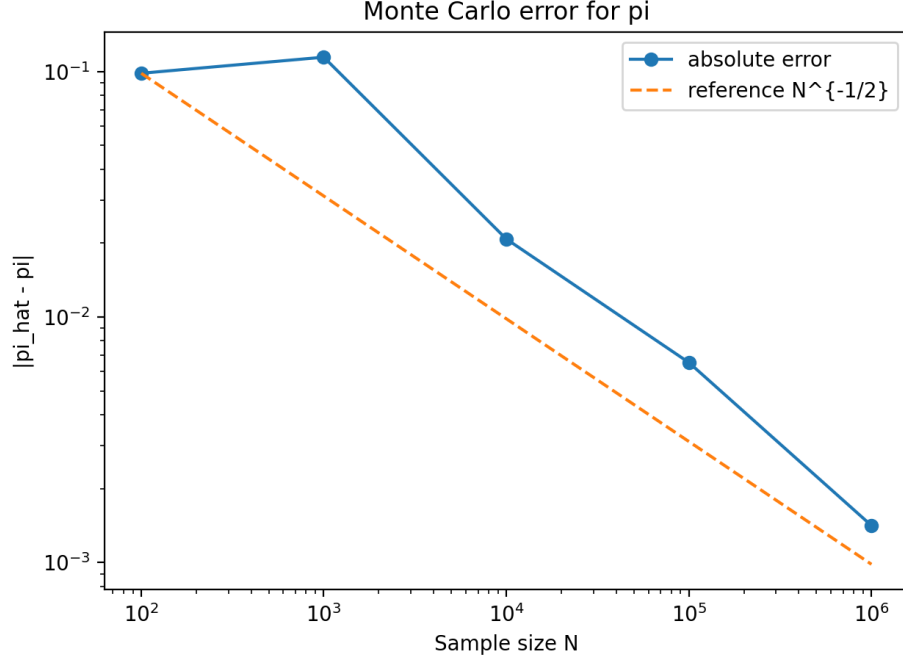


Figure 2: Absolute error of the Monte Carlo estimate of π . The dashed reference curve is proportional to $N^{-1/2}$.

The error does not decrease monotonically at every single sample size because the method is random. However, its typical magnitude decreases as N grows. The log-log plot is consistent with the standard Monte Carlo scaling

$$|\hat{\pi}_N - \pi| = O(N^{-1/2}). \quad (5)$$

3 Change of Variables: $Y = U^2$

Analytic density

Let

$$U \sim \text{Uniform}(0, 1), \quad Y = U^2. \quad (6)$$

For $0 < y < 1$, the inverse transformation is

$$u = \sqrt{y}. \quad (7)$$

Since $f_U(u) = 1$ on $(0, 1)$, the density of Y is

$$f_Y(y) = f_U(\sqrt{y}) \left| \frac{d}{dy} \sqrt{y} \right| = \frac{1}{2\sqrt{y}}, \quad 0 < y < 1. \quad (8)$$

Simulation result

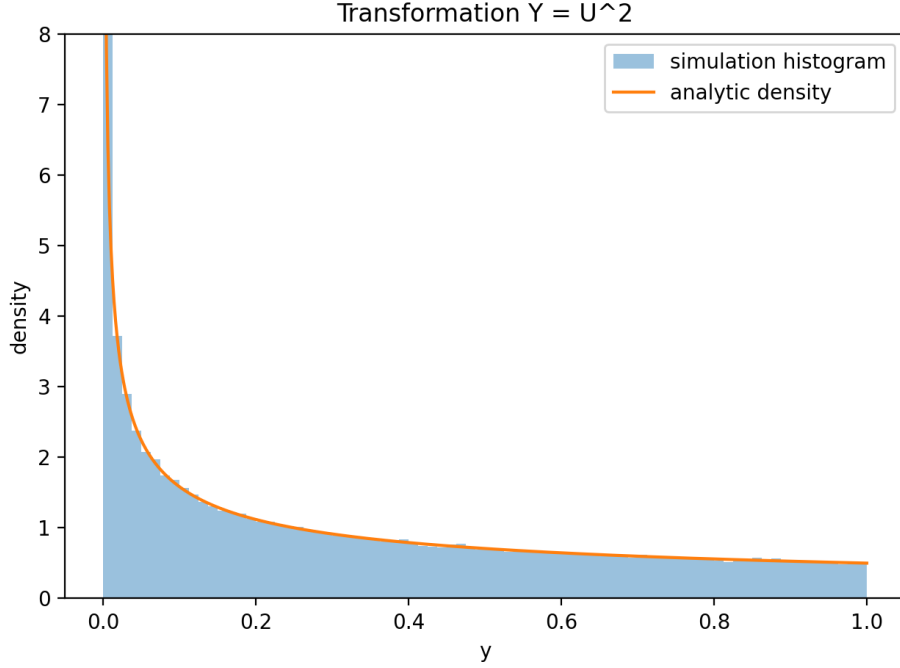


Figure 3: Histogram of $Y = U^2$ compared with the analytic density $f_Y(y) = 1/(2\sqrt{y})$.

The histogram is high near $y = 0$ and decreases as y increases. This agrees with the analytic density $1/(2\sqrt{y})$, which has an integrable singularity at $y = 0$.

4 Inverse Transform Sampling for the Exponential Distribution

Method

For an exponential random variable with rate $\lambda = 1.5$, the cumulative distribution function is

$$F_Y(y) = 1 - e^{-\lambda y}, \quad y \geq 0. \quad (9)$$

Using inverse transform sampling, set $F_Y(y) = U$, where $U \sim \text{Uniform}(0, 1)$. Then

$$U = 1 - e^{-\lambda y}, \quad (10)$$

$$e^{-\lambda y} = 1 - U, \quad (11)$$

$$y = -\frac{\ln(1 - U)}{\lambda}. \quad (12)$$

Thus the simulated exponential variable is

$$Y = -\frac{\ln(1 - U)}{1.5}. \quad (13)$$

The exact probability density function is

$$f_Y(y) = \lambda e^{-\lambda y} = 1.5e^{-1.5y}, \quad y \geq 0. \quad (14)$$

Simulation result

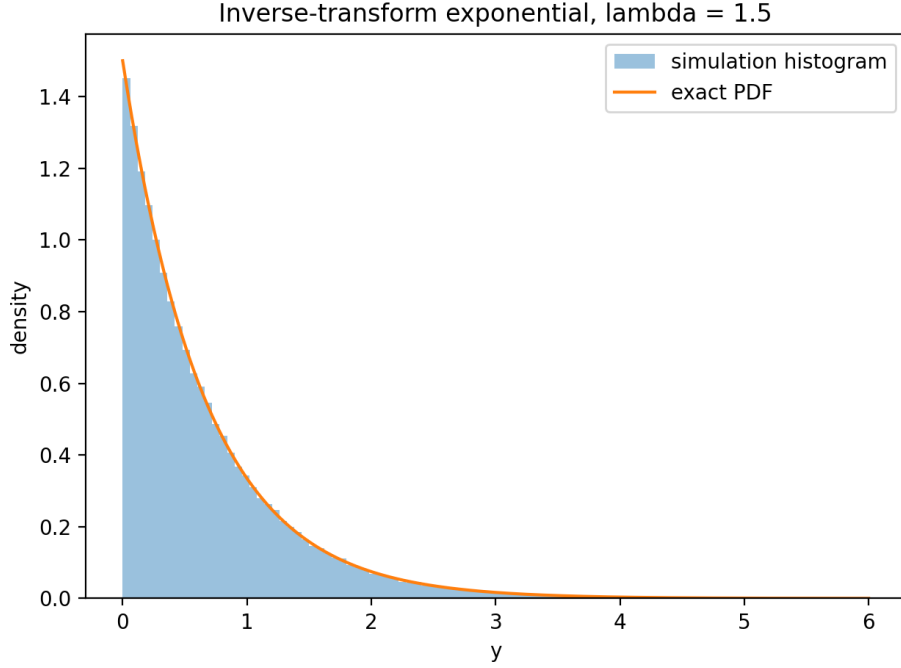


Figure 4: Histogram of the simulated exponential sample compared with the exact PDF $1.5e^{-1.5y}$.

Numerical check

The sample mean is

$$\bar{Y} \approx 0.664903. \quad (15)$$

The theoretical mean is

$$\mathbb{E}[Y] = \frac{1}{\lambda} = \frac{1}{1.5} \approx 0.666667. \quad (16)$$

The agreement is good, and the histogram follows the theoretical exponential density.

5 Empirical CDF

Method

For a sample $Y_1, \dots, Y_M$, the empirical cumulative distribution function is

$$\hat{F}_M(y) = \frac{1}{M} \sum_{i=1}^M \mathbf{1}\{Y_i \leq y\}. \quad (17)$$

For the exponential distribution with $\lambda = 1.5$, the exact CDF is

$$F_Y(y) = 1 - e^{-1.5y}, \quad y \geq 0. \quad (18)$$

Simulation result

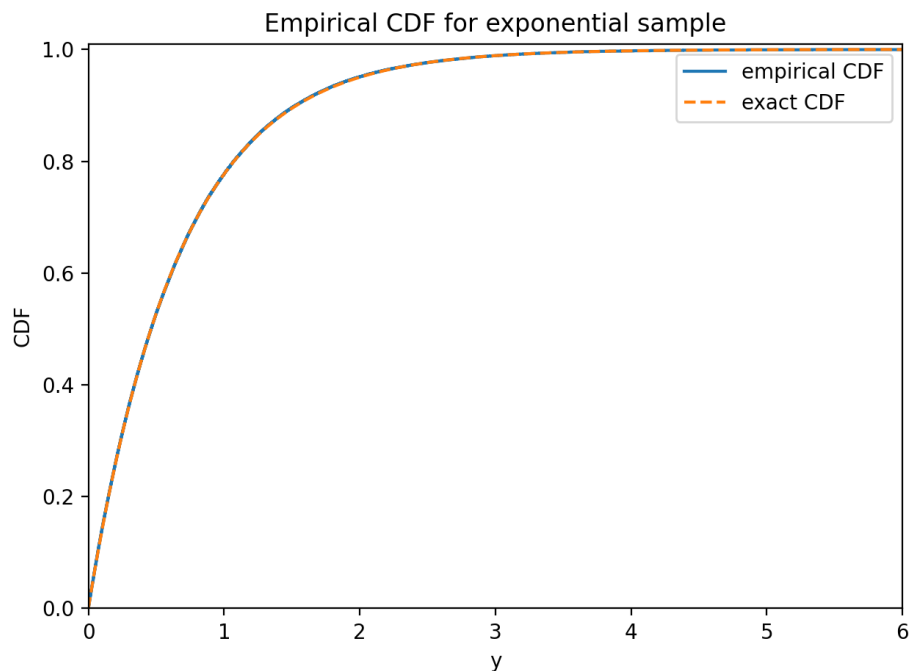


Figure 5: Empirical CDF of the exponential sample compared with the exact CDF $1 - e^{-1.5y}$.

The empirical CDF is close to the theoretical CDF. Small deviations are caused by finite-sample randomness. As the sample size increases, the empirical CDF converges uniformly to the true CDF according to the Glivenko–Cantelli theorem.

Code

The full Python code used to generate the results and figures is included below.

```
1 """
2 HomeWork 07 solution
3 Topics: Law of Large Numbers, Monte Carlo estimation, variable
4         transformation,
5         inverse transform sampling, and empirical CDF.
6 """
7 import math
8 import numpy as np
9 import matplotlib.pyplot as plt
10
11 # ----- Exercise 1: Coin tosses and LLN -----
12 rng = np.random.default_rng(20260511)
13 N = 100_000
14 tosses = rng.integers(0, 2, size=N) # 1=heads, 0=tails
15 running_fraction_heads = np.cumsum(tosses) / np.arange(1, N + 1)
16
17 print("Exercise 1: running fraction of heads")
18 for n in [10, 100, 1_000, 10_000, 100_000]:
19     frac = running_fraction_heads[n - 1]
20     print(f"n={n:>7,d}, fraction={frac:.6f}, deviation={frac - 0.5:+.6f}
21           ")
```

```

21 plt.figure()
22 plt.plot(np.arange(1, N + 1), running_fraction_heads)
23 plt.axhline(0.5, linestyle="--")
24 plt.xscale("log")
25 plt.xlabel("Number of tosses n")
26 plt.ylabel("Running fraction of heads")
27 plt.title("LLN for fair coin tosses")
28 plt.tight_layout()
29 plt.savefig("ex1_coin_lln.png", dpi=200)
30
31 # ----- Exercise 2: Monte Carlo estimate of pi
32 # -----
33 # Two independent RNG streams are used: one for x-coordinates and one
34 # for y-coordinates.
35 seed_sequence = np.random.SeedSequence(12345)
36 seq_x, seq_y = seed_sequence.spawn(2)
37 rng_x = np.random.default_rng(seq_x)
38 rng_y = np.random.default_rng(seq_y)
39
40 sample_sizes = np.array([100, 1_000, 10_000, 100_000, 1_000_000])
41 maxN = int(sample_sizes.max())
42 x = rng_x.random(maxN)
43 y = rng_y.random(maxN)
44 inside_cumsum = np.cumsum(x*x + y*y <= 1.0)
45
46 print("\nExercise 2: Monte Carlo estimate of pi")
47 pi_estimates = []
48 errors = []
49 for n in sample_sizes:
50     inside = inside_cumsum[n - 1]
51     pi_hat = 4 * inside / n
52     error = abs(pi_hat - math.pi)
53     pi_estimates.append(pi_hat)
54     errors.append(error)
55     print(f"N={n:>9,d}, inside={inside:>9,d}, pi_hat={pi_hat:.8f},
56           error={error:.8f}")
57
58 plt.figure()
59 plt.loglog(sample_sizes, errors, marker="o", label="absolute error")
60 reference = errors[0] * (sample_sizes / sample_sizes[0])**(-0.5)
61 plt.loglog(sample_sizes, reference, linestyle="--", label="reference N
62             ~{-1/2}")
63 plt.xlabel("Sample size N")
64 plt.ylabel("|pi_hat - pi|")
65 plt.title("Monte Carlo error for pi")
66 plt.legend()
67 plt.tight_layout()
68 plt.savefig("ex2_pi_error.png", dpi=200)
69
70 # ----- Exercise 3: Change of variables Y = U^2
71 # -----
72 rng3 = np.random.default_rng(222)
73 M = 200_000
74 U = rng3.random(M)
75 Y = U**2
76
77 y_grid = np.linspace(1e-4, 1.0, 500)
78 fY = 1 / (2 * np.sqrt(y_grid))

```

```

74
75 plt.figure()
76 plt.hist(Y, bins=80, density=True, alpha=0.45, label="simulation
    histogram")
77 plt.plot(y_grid, fY, label="analytic density")
78 plt.ylim(0, 8)
79 plt.xlabel("y")
80 plt.ylabel("density")
81 plt.title("Transformation  $Y = U^2$ ")
82 plt.legend()
83 plt.tight_layout()
84 plt.savefig("ex3_y_equals_u2.png", dpi=200)
85
86 print("\nExercise 3: The histogram follows  $f_Y(y)=1/(2 \sqrt{y})$  on  $0 < y < 1$ ." )
87
88 # ----- Exercise 4: Inverse transform exponential
    -----
89 lambda_value = 1.5
90 rng4 = np.random.default_rng(333)
91 U = rng4.random(M)
92 Yexp = -np.log(1 - U) / lambda_value
93
94 x_grid = np.linspace(0, 6, 500)
95 pdf = lambda_value * np.exp(-lambda_value * x_grid)
96
97 plt.figure()
98 plt.hist(Yexp, bins=100, range=(0, 6), density=True, alpha=0.45, label=
    "simulation histogram")
99 plt.plot(x_grid, pdf, label="exact PDF")
100 plt.xlabel("y")
101 plt.ylabel("density")
102 plt.title("Inverse-transform exponential,  $\lambda = 1.5$ ")
103 plt.legend()
104 plt.tight_layout()
105 plt.savefig("ex4_exp_pdf.png", dpi=200)
106
107 print("Exercise 4: sample mean =", np.mean(Yexp), "; theoretical mean =
    ", 1/lambda_value)
108
109 # ----- Exercise 5: Empirical CDF -----
110 ys = np.sort(Yexp)
111 ecdf = np.arange(1, M + 1) / M
112 cdf_exact = 1 - np.exp(-lambda_value * x_grid)
113
114 plt.figure()
115 idx = np.linspace(0, M - 1, 3000, dtype=int)
116 plt.step(ys[idx], ecdf[idx], where="post", label="empirical CDF")
117 plt.plot(x_grid, cdf_exact, linestyle="--", label="exact CDF")
118 plt.xlim(0, 6)
119 plt.ylim(0, 1.01)
120 plt.xlabel("y")
121 plt.ylabel("CDF")
122 plt.title("Empirical CDF for exponential sample")
123 plt.legend()
124 plt.tight_layout()
125 plt.savefig("ex5_exp_ecdf.png", dpi=200)
126

```



```
127 print("Exercise 5: The empirical CDF is close to  $F(y)=1-\exp(-1.5y)$ .")
128 plt.show()
```